NLP-Powered Conversational AI Builder Platform with text analysis

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology

in The Department of AI&DS

Text Analytics with 23SDCS18A

Submitted by

2310080023: NACHU GOWTHAM

2310080029: MARUPAKA SNIGDHA SHARMA

Under the guidance of

MANI KUMAR

Department of Electronics and Communication Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

April- 2026.

Abstract

The NLP-Powered Conversational AI Builder Platform with Text Analysis is developed as a full-stack system that enables users to create, train, configure, and deploy knowledge-grounded chatbots through a unified workflow. The final implementation is designed to solve a practical deployment problem in conversational AI: most chatbot systems provide fluent generation but lack robust grounding, configurable conversation control, and reliable runtime behavior under real operational constraints. This platform addresses that gap by combining management-plane controls for bot owners, runtime-plane access for website visitors, and a backend orchestration pipeline that connects ingestion, preprocessing, retrieval, response generation, and persistent state management. The result is a builder-oriented architecture where non-ML users can deploy domain-specific assistants that respond from ingested content rather than unconstrained model memory.

The system architecture is composed of a Next.js management frontend, a FastAPI backend for orchestration and API routing, a dedicated embedding microservice for vector generation, and a Supabase PostgreSQL layer with vector support for semantic retrieval. This separation is intentional: backend orchestration remains lightweight and policy-driven while embedding inference is isolated for scalability and independent deployment. The backend supports bot lifecycle management, API-key-secured runtime chat access, document and website knowledge ingestion, session-aware conversation handling, and flow-based dialog execution. A visual flow model allows configurable interaction behavior beyond simple question-answer patterns, while an embeddable JavaScript widget enables direct integration into external websites with minimal setup effort.

The NLP and retrieval core is implemented as a staged pipeline rather than a direct model call. User input is sanitized and normalized, intent and context signals are derived, and retrieval queries are transformed before search. Knowledge retrieval follows a hybrid strategy that combines dense vector matching with lexical relevance support and ranking stabilization. Retrieved candidates are filtered and assembled into controlled context before response generation. This design improves grounding consistency, especially in short follow-up queries, entity-sensitive prompts, and mixed conversational contexts where semantic-only ranking may underperform. The response stage executes through failover-aware provider routing, and all interactions are persisted with session continuity to support multi-turn conversational coherence.

The final outcome demonstrates that practical conversational AI quality depends on orchestration discipline across the entire pipeline, not solely on model selection. By integrating structured ingestion, hybrid retrieval, context-controlled generation, persistent session logic, and secure multi-tenant boundaries, the platform reaches a production-aligned maturity level for its defined scope. Future work will extend this foundation through quantitative retrieval and grounding evaluation, larger-scale concurrency testing, richer observability instrumentation, and advanced corpus diagnostics to further improve trustworthiness and deployment readiness.

.

NLP-Powered Conversational AI Builder Platform with Text Analysis

1. Introduction

The final report stage addresses the real deployment challenge in conversational AI: building systems that remain grounded, controllable, and reliable under variable user input and constrained infrastructure. Many chatbot solutions can produce fluent answers, but fewer provide a complete architecture where ingestion quality, retrieval relevance, conversation control, security boundaries, and operational resilience are designed as one pipeline. This project resolves that gap by delivering a full-stack builder platform that separates management operations from public runtime interaction while preserving shared retrieval intelligence. The management side allows owners to define chatbot behavior, train with uploaded or scraped content, configure appearance, and control flow logic. The runtime side allows external website users to interact through an embedded widget secured by bot-scoped authorization. The final increment consolidates these capabilities into a stable end-to-end system where NLP, NLU, retrieval, generation, and persistence are coordinated, observable, and failure-tolerant.

2. SYSTEM ARCHITECTURE

The platform architecture is organized into management, orchestration, inference, and persistence domains.

Layer/Domain	Main Responsibility	Key Output
Frontend Management Domain	Bot creation, configuration, ingestion actions, flow design, appearance settings	Authenticated control commands
Runtime Widget Domain	End-user interaction from external websites	Bot-key scoped chat requests
Backend Orchestration Domain	API routing, auth checks, ingestion, retrieval, flow execution, response handling	Structured chat/inference pipeline
Embedding Inference Domain	Text-to-vector generation	384-dimensional embeddings
Persistence Domain	Bot metadata, chunk corpus, vectors, sessions, message history	Durable multi-tenant state

The architecture separates control-plane operations (owner actions) from runtime-plane operations (public chat execution), while keeping both paths connected to a shared knowledge and session backend.

2.1 High-Level Runtime Topology

```
Owner Dashboard  --------\
                          -> Main Backend API -> Supabase/PostgreSQL (pgvector)
Website Widget  ---------/          |
                                     -> Embedding Microservice
```

2.2 Backend Layered Architecture

Backend Layer	Technical Role
API Layer	Receives and routes HTTP requests across health, bot, and chat domains
Access Control Layer	Enforces user JWT for owner routes and bot API key for runtime routes
Validation Layer	Applies schema-level checks on payloads and flow structure

Backend Layer	Technical Role
Orchestration Layer	Executes ingestion and conversation logic across services
Integration Layer	Connects external services (embedding, crawling, generation)
Persistence Layer	Stores and retrieves bots, documents, chunks, vectors, sessions, and messages

3. DATABASE DESIGN AND SCHEMA

The database is designed for both relational integrity and retrieval performance. Bot ownership, knowledge indexing, and conversation continuity are stored in separate but linked tables.

Table	Purpose
profiles	User identity extension for authenticated users
bots	Bot configuration, API key, prompt, flow JSON, appearance JSON
documents	Source metadata for uploaded/scraped knowledge
document_chunks	Retrieval units with chunk content and embedding vectors
chat_messages	Immutable user-assistant interaction history
chat_sessions	Mutable runtime state for flow continuity

Vector retrieval is supported using pgvector with indexed cosine operations. This enables semantic nearest-neighbor search over chunk embeddings while preserving bot-level data scoping.

4. METHODOLOGY

The final-phase methodology follows a system-centric iterative model. Instead of optimizing only model output, the development approach optimized each stage of the conversational pipeline: ingestion quality, query preprocessing, retrieval robustness, response grounding, and runtime fault tolerance. The methodology also emphasized deterministic transformation from raw source text to indexed retrieval units and from raw user query to retrieval-ready query form. Reliability engineering was treated as part of core functionality, not post-processing, by integrating fallback behavior, state validation, and staged progress observability into the main execution path.

5. NLP, NLU, RETRIEVAL DESIGN

Subsystem	Functional Contribution
NLU preprocessing	Improves query interpretability and follow-up handling
Dense retrieval	Captures semantic similarity with indexed chunks
Lexical retrieval	Preserves exact-term/entity relevance
Rank fusion and filtering	Improves evidence stability and reduces noisy context
Prompt assembly	Ensures generation receives structured grounded context

6. EXPERIMENTS

Final-stage experiments are integration-driven and evaluate complete request lifecycles rather than isolated component outputs. Validation was performed across API boundaries, ingestion flow, retrieval behavior, runtime conversation continuity, and fallback behavior under degraded dependency conditions. The experiments confirm that the system executes as a coordinated pipeline and preserves grounded conversational behavior through persisted session and knowledge state.

7. RESULTS

The final increment delivers a complete architecture where management and runtime paths are operationally unified but securely separated. Retrieval behavior is stronger due to hybrid evidence handling and controlled context assembly. Ingestion is consistent across file and web sources. Runtime reliability is improved through fallback-aware processing and session-state safety. The platform now demonstrates system-level readiness for real usage under constrained cloud resources, with clear paths for future quantitative benchmarking and scale testing.

Outcome Area	Final Status
End-to-end orchestration	Completed and operational
Hybrid retrieval pipeline	Implemented and active
Flow + session continuity	Implemented and persistent
Ingestion observability	Implemented with streamed stages
Resilience mechanisms	Implemented with fallback and failover behavior

8. CONCLUSION AND FUTURE WORK

The final increment confirms that the NLP-Powered Conversational AI Builder Platform with Text Analysis has reached architectural completeness for its defined scope. The system now functions as a coherent conversational AI platform where ingestion, NLP/NLU processing, retrieval, response generation, and persistence are tightly integrated. The most important achievement is operational reliability through explicit orchestration and fallback-aware execution. Future work should focus on deeper quantitative evaluation, large-scale concurrency testing, richer observability metrics, retrieval explainability signals, and expanded corpus quality diagnostics to further improve trustworthiness and production scale-readiness.

REFERENCES

[1] FastAPI Documentation, FastAPI.

[2] Supabase Documentation, Supabase.

[3] PostgreSQL Documentation, PostgreSQL Global Development Group.

[4] pgvector Documentation and Repository.

[5] Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.

[6] Reimers and Gurevych, “Sentence-BERT,” EMNLP-IJCNLP, 2019.

[7] Robertson and Zaragoza, “BM25 and Beyond,” Foundations and Trends in IR, 2009.

[8] Pedregosa et al., “Scikit-learn: Machine Learning in Python,” JMLR, 2011.

[9] Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” EMNLP System Demonstrations, 2020.

[10] spaCy Documentation.

[11] tiktoken Repository.

[12] Playwright Documentation.

[13] Crawl4AI Documentation.

[14] Groq API Documentation.